Trabajo práctico: Verificación de software

Ingeniería del software II

Aldana Zarate

Abril 2026

Descripción del problema: Sistema de inscripciones a torneos de natación

Se necesita la creación de un sistema de inscripciones a torneos para la Federación Rosario de Natación. En este sistema se debe poder anotar un nadador a una carrera que ya posea árbitros, asignar árbitros a una carrera y mostrar los nadadores inscriptos y árbitros de una carrera del torneo.

Designaciones

- n es un nadador de la FRN $\approx n \in NADADOR$
- a es un árbitro de la FRN $\approx a \in ARBITRO$
- c es una carrera del torneo $\approx c \in CARRERA$
- Conjunto de nadadores inscriptos a una carrera $c \approx nadadoresInscriptos(c)$
- Conjunto de árbitros a supervisar una carrera $c \approx arbitrosAsignados(c)$

Especificación Z

Definiciones de tipo [1]

$[NADADOR, ARBITRO, CARRERA]$

Esquemas de estado

Torneo

$nadadoresInscriptos : CARRERA \rightarrowtail \mathbb{P}\, NADADOR$
$arbitrosAsignados : CARRERA \rightarrowtail \mathbb{P}\, ARBITRO$

TorneoInicial

Torneo

$nadadoresInscriptos = \emptyset$
$arbitrosAsignados = \emptyset$

Invariante

Del requerimiento "se debe poder anotar un nadador a una carrera que ya posea árbitros"tenemos el siguiente invariante: un nadador no se puede inscribir a una carrera que todavía no tenga árbitros asignados.

TorneoInv

Torneo

$\mathrm{dom}(nadadoresInscriptos) \subseteq \mathrm{dom}(arbitrosAsignados)$

Operaciones

Anotar un nadador a una carrera [2]

[1] No fue necesario definir constantes.

[2] Nota sobre el dominio de aplicación: cuando se asignan los árbitros, no se agrega la carrera a *nadadoresInscriptos* ya que la idea es que esta función solo contenga los datos de los inscriptos. En consecuencia, por eso hay dos casos a la hora de inscribir un nadador y uno para devolver al conjunto vacío si se pregunta por una carrera que no esté en *nadadoresInscriptos*, ya que es necesario que este tipo de fenómenos quede manifiesto. Se podría haber manejado toda la información en una sola función y que su conjunto imagen sea un producto cartesiano entre *ARBITRO* y *NADADOR* pero esta forma me pareció más sencilla.

$InscribirNadadorHayCompetidoresOk$

$\Delta Torneo$
$n? : NADADOR$
$c? : CARRERA$

$c? \in \mathrm{dom}(arbitrosAsignados)$
$c? \in \mathrm{dom}(nadadoresInscriptos)$
$n? \notin nadadoresInscriptos(c?)$
$nadadoresInscriptos' = nadadoresInscriptos \oplus \{c? \mapsto (nadadoresInscriptos(c?) \cup \{n?\})\}$
$arbitrosAsignados' = arbitrosAsignados$

$InscribirNadadorSinCompetidoresOk$

$\Delta Torneo$
$n? : NADADOR$
$c? : CARRERA$

$c? \in \mathrm{dom}(arbitrosAsignados)$
$c? \notin \mathrm{dom}(nadadoresInscriptos)$
$nadadoresInscriptos' = nadadoresInscriptos \cup \{c? \mapsto \{n?\}\}$
$arbitrosAsignados' = arbitrosAsignados$

$NadadorYaInscripto$

$\Xi Torneo$
$n? : NADADOR$
$c? : CARRERA$

$c? \in \mathrm{dom}(arbitrosAsignados)$
$c? \in \mathrm{dom}(nadadoresInscriptos)$
$n? \in nadadoresInscriptos(c?)$
$nadadoresInscriptos' = nadadoresInscriptos$
$arbitrosAsignados' = arbitrosAsignados$

$CarreraNoHabilitada$

$\Xi Torneo$
$n? : NADADOR$
$c? : CARRERA$

$c? \notin \mathrm{dom}(arbitrosAsignados)$
$c? \notin \mathrm{dom}(nadadoresInscriptos)$
$nadadoresInscriptos' = nadadoresInscriptos$
$arbitrosAsignados' = arbitrosAsignados$

$InscribirNadador \mathrel{\widehat{=}} InscribirNadadorHayCompetidoresOk \lor InscribirNadadorSinCompetidoresOk \lor NadadorYaInscripto$
$\lor CarreraNoHabilitada$

Asignar árbitros a una carrera

$AsignarArbitrosOk$

$\Delta Torneo$
$ca? : \mathbb{P}\, ARBITRO$
$c? : CARRERA$

$c? \notin \mathrm{dom}(arbitrosAsignados)$
$c? \notin \mathrm{dom}(nadadoresInscriptos)$
$nadadoresInscriptos' = nadadoresInscriptos$
$arbitrosAsignados' = arbitrosAsignados \cup \{c? \mapsto ca?\}$

$$
\begin{array}{|l}
\hline
\textit{CarreraYaHabilitada} \\
\Xi Torneo \\
ca? : \mathbb{P}\, ARBITRO \\
c? : CARRERA \\
\hline
c? \in \mathrm{dom}(arbitrosAsignados) \\
nadadoresInscriptos' = nadadoresInscriptos \\
arbitrosAsignados' = arbitrosAsignados \\
\hline
\end{array}
$$

$$AsignarArbitros \mathrel{\widehat{=}} AsignarArbitrosOk \lor CarreraYaHabilitada$$

Mostrar los nadadores inscriptos y árbitros de una carrera

$$
\begin{array}{|l}
\hline
\textit{MostrarInfoCarreraHayInscriptosOk} \\
\Xi Torneo \\
c? : CARRERA \\
ni! : \mathbb{P}\, NADADOR \\
aa! : \mathbb{P}\, ARBITRO \\
\hline
c? \in \mathrm{dom}(arbitrosAsignados) \\
c? \in \mathrm{dom}(nadadoresInscriptos) \\
ni! = nadadoresInscriptos(c?) \\
aa! = arbitrosAsignados(c?) \\
\hline
\end{array}
$$

$$
\begin{array}{|l}
\hline
\textit{MostrarInfoCarreraSinInscriptosOk} \\
\Xi Torneo \\
c? : CARRERA \\
ni! : \mathbb{P}\, NADADOR \\
aa! : \mathbb{P}\, ARBITRO \\
\hline
c? \in \mathrm{dom}(arbitrosAsignados) \\
c? \notin \mathrm{dom}(nadadoresInscriptos) \\
ni! = \emptyset \\
aa! = arbitrosAsignados(c?) \\
\hline
\end{array}
$$

$$
\begin{array}{|l}
\hline
\textit{NoHayInfoCarrera} \\
\Xi Torneo \\
c? : CARRERA \\
ni! : \mathbb{P}\, NADADOR \\
aa! : \mathbb{P}\, ARBITRO \\
\hline
c? \notin \mathrm{dom}(arbitrosAsignados) \\
c? \notin \mathrm{dom}(nadadoresInscriptos) \\
ni! = \emptyset \\
aa! = \emptyset \\
\hline
\end{array}
$$

$$MostrarInfoCarrera \mathrel{\widehat{=}} MostrarInfoCarreraHayInscriptosOk \lor MostrarInfoCarreraSinInscriptosOk \lor NoHayInfoCarrera$$

Finalmente, se pasó la especificación por `z-eves` y pasó exitosamente el control de tipos.

Traducción de la especificación Z a un programa {log}

Dado que hemos dado uso a algunos conjuntos en lugar de tipos para algunas variables de estado, resulta conveniente normalizar la especificación Z antes de realizar la traducción. Habría que modificar el tipo esquema eliminando el uso de funciones parciales:

$$
\begin{array}{|l}
\hline
\textit{Torneo} \\
\textit{nadadoresInscriptos} : \textit{CARRERA} \leftrightarrow \mathbb{P}\,\textit{NADADOR} \\
\textit{arbitrosAsignados} : \textit{CARRERA} \leftrightarrow \mathbb{P}\,\textit{ARBITRO} \\
\hline
\end{array}
$$

Y agregando dichas restricciones en el invariante:

$$
\begin{array}{|l}
\hline
\textit{TorneoInv} \\
\textit{Torneo} \\
\hline
\mathrm{dom}(\textit{nadadoresInscriptos}) \subseteq \mathrm{dom}(\textit{arbitrosAsignados}) \\
\textit{nadadoresInscriptos} \in \textit{CARRERA} \nrightarrow \mathbb{P}\,\textit{NADADOR} \\
\textit{arbitrosAsignados} \in \textit{CARRERA} \nrightarrow \mathbb{P}\,\textit{ARBITRO} \\
\hline
\end{array}
$$

La traducción a $\{log\}$ se puede encontrar en el archivo `sistemaTorneos.slog`, la cual fue aceptada satisfactoriamente por el typechecker.

Una observación es que este paso de traducir la especificación a $\{log\}$ permitió darme cuenta de un error de atención que había tenido al hacer rápido la especificación quizás, al poner en una de las operaciones algo imposible como $c? \notin dom(nadadoresInscriptos) \wedge n? \notin nadadoresInscriptos(c?)$, ya que por la primera condición, es trivial ver que no sería posible calcular $nadadoresInscriptos(c?)$.

Uso del VCG para generar las condiciones de verificación

Al ejecutar el comando `vcg('sistemaTorneos.slog').` noté que tenía 2 errores y 2 warnings. Los errores eran a causa de que en las operaciones que especificaban casos de error, tenía parámetros de entrada que no usaba luego. Esto me causó un error en *carreraNoHabilitada* (pasaba como parámetro un *Nadador* por el solo hecho de que tenga el mismo tipo que la operación total) y en *carreraYaHabilitada* (pasaba un conjunto de *Arbitros* para respetar el tipo de la operación total, y luego no lo usaba en el resto de la operación.)

También me hizo dar cuenta de un error de tipeo en la definición del invariante *torneoPfunAAInv*, ya que por copiar y pegar el primer invariante para definir el segundo debido a sus similitudes, había olvidado cambiar 2 veces *NI* por *AA*.

Luego de hacer estas modificaciones, se pudo generar correctamente el archivo `sistemaTorneos-vc.slog`.

Ejecución de simulaciones

Pasamos a ejecutar 2 simulaciones sobre el prototipo usando el entorno Next, mostrando la traza de ejecución de cada una:

Simulación 1

```
[initial]:[torneoDominiosInv, torneoPfunNIInv, torneoPfunAAInv] >>
asignarArbitros(Arbitros:{a1,a2,a3,a4,a5}, Carrera:c1) >>
inscribirNadador(Nadador:[[n1,n2,n3]], Carrera:c1) >>
mostrarInfoCarrera(Carrera:c1).

Execution trace is:
NadadoresInscriptos = {},
ArbitrosAsignados = {}
  ----> asignarArbitros(Arbitros:({}with a5 with a4 with a3 with a2 with a1),Carrera:c1)
NadadoresInscriptos = {},
ArbitrosAsignados = {[c1,{a1,a2,a3,a4,a5}]}
  ----> inscribirNadador(Nadador:n1,Carrera:c1)
NadadoresInscriptos = {[c1,{n1}]},
ArbitrosAsignados = {[c1,{a1,a2,a3,a4,a5}]}
  ----> inscribirNadador(Nadador:n2,Carrera:c1)
NadadoresInscriptos = {[c1,{n2,n1}]},
ArbitrosAsignados = {[c1,{a1,a2,a3,a4,a5}]}
  ----> inscribirNadador(Nadador:n3,Carrera:c1)
```

```
NadadoresInscriptos = {[c1,{n3,n2,n1}]},
ArbitrosAsignados = {[c1,{a1,a2,a3,a4,a5}]}
  ----> mostrarInfoCarrera(Carrera:c1)
S_Nadadores = {n3,n2,n1},
S_Arbitros = {a1,a2,a3,a4,a5},
NadadoresInscriptos = {[c1,{n3,n2,n1}]},
ArbitrosAsignados = {[c1,{a1,a2,a3,a4,a5}]}

true
```

Observamos que la simulación chequea los invariantes y se obtiene el resultado esperado. La idea de la misma es simular un escenario de uso *normal* del sistema (el cual usa los casos de éxito de las operaciones): Se le asigna un conjunto de árbitros a una carrera, luego van 3 nadadores a inscribirse a la misma y finalmente se muestra la información disponible sobre la carrera en cuestión.

Segunda simulación

```
[initial]:[torneoDominiosInv, torneoPfunNIInv, torneoPfunAAInv] >>
inscribirNadador(Nadador:n1, Carrera:c1) >>
mostrarInfoCarrera(Carrera:c1) >>
asignarArbitros(Arbitros:{a1,a2,a3,a4,a5}, Carrera:c1) >>
asignarArbitros(Arbitros:{a6,a7,a8,a9,10}, Carrera:c1) >>
mostrarInfoCarrera(Carrera:c1) >>
inscribirNadador(Nadador:n1, Carrera:c1) >>
inscribirNadador(Nadador:n1, Carrera:c1) >>
mostrarInfoCarrera(Carrera:c1).

Execution trace is:
NadadoresInscriptos = {},
ArbitrosAsignados = {}
  ----> inscribirNadador(Nadador:n1,Carrera:c1)
NadadoresInscriptos = {},
ArbitrosAsignados = {}
  ----> mostrarInfoCarrera(Carrera:c1)
S_Nadadores = {},
S_Arbitros = {},
NadadoresInscriptos = {},
ArbitrosAsignados = {}
  ----> asignarArbitros(Arbitros:({}with a5 with a4 with a3 with a2 with a1),Carrera:c1)
S_Nadadores = {},
S_Arbitros = {},
NadadoresInscriptos = {},
ArbitrosAsignados = {[c1,{a1,a2,a3,a4,a5}]}
  ----> asignarArbitros(Arbitros:({}with 10 with a9 with a8 with a7 with a6),Carrera:c1)
S_Nadadores = {},
S_Arbitros = {},
NadadoresInscriptos = {},
ArbitrosAsignados = {[c1,{a1,a2,a3,a4,a5}]}
  ----> mostrarInfoCarrera(Carrera:c1)
S_Nadadores = {},
S_Arbitros = {a1,a2,a3,a4,a5},
NadadoresInscriptos = {},
ArbitrosAsignados = {[c1,{a1,a2,a3,a4,a5}]}
  ----> inscribirNadador(Nadador:n1,Carrera:c1)
S_Nadadores = {},
S_Arbitros = {a1,a2,a3,a4,a5},
NadadoresInscriptos = {[c1,{n1}]},
ArbitrosAsignados = {[c1,{a1,a2,a3,a4,a5}]}
```

```
  ----> inscribirNadador(Nadador:n1,Carrera:c1)
S_Nadadores = {},
S_Arbitros = {a1,a2,a3,a4,a5},
NadadoresInscriptos = {[c1,{n1}]},
ArbitrosAsignados = {[c1,{a1,a2,a3,a4,a5}]}
  ----> mostrarInfoCarrera(Carrera:c1)
S_Nadadores = {n1},
S_Arbitros = {a1,a2,a3,a4,a5},
NadadoresInscriptos = {[c1,{n1}]},
ArbitrosAsignados = {[c1,{a1,a2,a3,a4,a5}]}

true
```

Observamos que en esta simulación también se chequean los invariantes y se obtiene el resultado esperado. La idea de esta simulación es probar las operaciones que especifican casos de error de las operaciones para contemplar estos casos *anormales* de uso: un nadador se quiere inscribir a una carrera no habilitada, se pide mostrar información de una carrera inexistente, se quieren asignar árbitros a una carrera que ya los posee, se pide mostrar información sobre una carrera que no tiene inscriptos y se quiere inscribir dos veces un mismo nadador a una misma carrera.

Descarga de todas las condiciones de verificación generadas por el VCG

En un primer intento de descarga al ejecutar `check_vcs_sistemaTorneos.` se obtuvo el siguiente resultado:

```
{log}=> check_vcs_sistemaTorneos.

Checking torneoInicial_sat_torneoDominiosInv ... OK
Checking torneoInicial_sat_torneoPfunNIInv ... OK
Checking torneoInicial_sat_torneoPfunAAInv ... OK
Checking inscribirNadador_is_sat ... OK
Checking asignarArbitros_is_sat ... OK
Checking mostrarInfoCarrera_is_sat ... ERROR
Checking inscribirNadador_pi_torneoDominiosInv ... OK
Checking inscribirNadador_pi_torneoPfunNIInv ... OK
Checking inscribirNadador_pi_torneoPfunAAInv ... OK
Checking asignarArbitros_pi_torneoDominiosInv ... OK
Checking asignarArbitros_pi_torneoPfunNIInv ... OK
Checking asignarArbitros_pi_torneoPfunAAInv ... OK
Checking mostrarInfoCarrera_pi_torneoDominiosInv ... OK
Checking mostrarInfoCarrera_pi_torneoPfunNIInv ... OK
Checking mostrarInfoCarrera_pi_torneoPfunAAInv ... OK

Total VCs: 15 (discharged: 14, failed: 1, timeout: 0)
Execution time (excluding timeouts): 0.14149045944213867 s
yes
```

Intenté ejecutar `vcgce(mostrarInfoCarrera_is_sat).` pero no dio mucha información:

```
{log}=> vcgce(mostrarInfoCarrera_is_sat).
findh error: it seems that mostrarInfoCarrera_is_sat has been proved

yes
```

Ante este error se procedió por:

- Reemplazar el uso de `let` en operaciones por `dec`.
- Eliminar las variables de estado primadas de las interfaces de la operación `mostrarInfoCarrera`, la cual no produce cambios en dichas variables.

Luego de hacer esto, `check_vcs_sistemaTorneos.` funcionó correctamente y devolvió `OK` para todos los casos.

Generación de casos de prueba a partir del programa {*log*} usando {*log*}-TTF

A lo largo de este trabajo, se han realizado las siguientes acciones:

- La especificación ha pasado el control de tipos.
- La especificación ha sido analizada por el VCG.
- El archivo con las condiciones de verificación ha sido consultado.
- Se han descargado las condiciones de verificación generadas por el VCG.

con lo cual procederemos a elaborar casos de testing para todas las operaciones.

Operación inscribirNadador

A continuación se deja la secuencia de comandos utilizados para obtener los casos de prueba.[3] Los mismos se pueden encontrar en el archivo `sistemaTorneos_inscribirNadador-tt.slog`.[4]

Comandos utilizados

```
ttf(sistemaTorneos).
applydnf(inscribirNadador(Nadador, Carrera)).
applysp(inscribirNadador_dnf_1, un(NICarr, {Nadador}, UnionNadadores)).
prunett.
applysp(inscribirNadador_dnf_1, oplus(NadadoresInscriptos, {[Carrera, UnionNadadores]}, NadadoresInscriptos_)).
prunett.
applysc(inscribirNadador_dnf_1, NadadoresInscriptos).
prunett.
applysp(inscribirNadador_dnf_2, un(NadadoresInscriptos, {[Carrera, {Nadador}]}, NadadoresInscriptos_)).
prunett.
applysc(inscribirNadador_dnf_2, NadadoresInscriptos).
prunett.
applysc(inscribirNadador_dnf_3, NadadoresInscriptos).
prunett.
applysc(inscribirNadador_dnf_4, ArbitrosAsignados).
prunett.
gentc.
writetc.
exporttt.
```

Primero se aplicó la táctica FND, la cual particiona la operación según las precondiciones de las suboperaciones que la conforman. Esto generó 4 nodos:

- `inscribirNadador_dnf_1`
- `inscribirNadador_dnf_2`
- `inscribirNadador_dnf_3`
- `teinscribirNadador_dnf_4`

uno por cada suboperación.

Luego, se aplica la táctica SP de los operadores `un` y `oplus` sobre `inscribirNadador_dnf_1`, correspondiente al caso exitoso de inscripción en una carrera con otros competidores, para testear el comportamiento de estos operadores. Finalmente para este nodo, se aplicó la táctica de SC sobre el conjunto `NadadoresInscriptos`, para tener varios escenarios posibles del conjunto sobre el cual se va a trabajar al final de la operación.

Sobre el nodo `inscribirNadador_dnf_2`, se aplica la táctica SP para testear el operador de unión usado en la suboperación y de manera análoga también se aplicó la técnica de SC.

Sobre los dos nodos restantes, se aplicó la técnica de SC sobre los conjuntos que analiza cada suboperación para dar lugar al caso que representa.

Entre cada técnica se aplicó `prunett` para podar las hojas insatisfacibles del árbol.

[3] En los listados de comandos se omitió la ejecución de varios *writett* (usados para ir visualizando el árbol) y un *exporttt* luego de aplicar la técnica de FND, usado para poder ver qué escenario correspondía a cada nodo generado por la técnica.

[4] Analizando los casos de prueba generados, pude observar que hay algunos que no cumplen con el invariante definido.

Operación asignarArbitros

A continuación se deja la secuencia de comandos utilizados para obtener los casos de prueba. Los mismos se pueden encontrar en el archivo `sistemaTorneos_asignarArbitros-tt.slog`.

Comandos utilizados

```
ttf(sistemaTorneos).
applydnf(asignarArbitros(Arbitros, Carrera)).
applysp(asignarArbitros_dnf_1, un(ArbitrosAsignados, {[Carrera, Arbitros]}, ArbitrosAsignados_)).
prunett.
applysc(asignarArbitros_vis, ArbitrosAsignados).
prunett.
gentc.
writetc.
exporttt.
```

En esta operación se aplicó la táctica FND, la cual creó un árbol con dos nodos, una para cada suboperación. Luego se aplicó la táctica SP sobre el nodo de asignación exitosa de árbitros, para testear el operador de unión. Finalmente se aplicó la táctica de SC en todo el árbol sobre el conjunto `ArbitrosAsignados`, el cual es el que manipula la operación.

Operación mostrarInfoCarrera

A continuación se deja la secuencia de comandos utilizados para obtener los casos de prueba. Los mismos se pueden encontrar en el archivo `sistemaTorneos_mostrarInfoCarrera-tt.slog`.

Comandos utilizados

```
ttf(sistemaTorneos).
applydnf(mostrarInfoCarrera(Carrera)).
applysc(mostrarInfoCarrera_vis, ArbitrosAsignados).
prunett.
applysc(mostrarInfoCarrera_vis, NadadoresInscriptos).
prunett.
gentc.
writetc.
exporttt.
```

En esta operación se aplicó la táctica FND, la cual creó un árbol de 3 nodos, uno para cada suboperación. Luego se aplicó la táctica SC sobre todo el árbol, primero con el conjunto `ArbitrosAsignados` y luego con `NadadoresInscriptos`.

Observando los casos de prueba generados, quizás resulte un poco excesivo aplicar esta táctica sobre todo el árbol, ya que los casos generados para el último nodo pueden ser un poco redundantes.